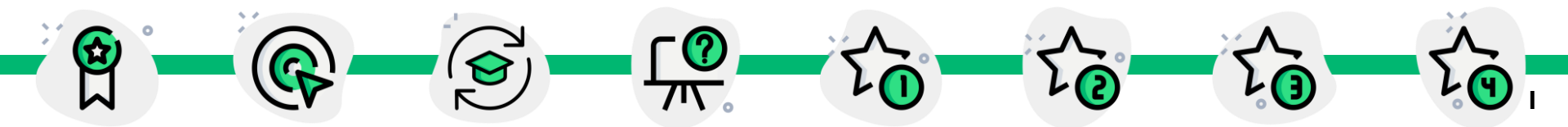


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.03.23

3주차



검색 알고리즘

03-1 검색 알고리즘이란?

03-2 선형 검색

03-3 이진 검색



03-1 검색 알고리즘이란?

검색 알고리즘이란?

- 데이터 집합에서 원하는 값을 가진 요소를 찾아내는 알고리즘을 검색 알고리즘이라고 함.

◆ 검색과 키 살펴보기

- 주소록 검색 : 예를 들어 국적을 검색하는 경우 국적이 “키(key)” 이고 나이를 검색하는 경우 나이가 키임.



03-1 검색 알고리즘이란?

◆ 배열에서 검색하기

- 선형 검색: 무작위로 늘어놓은 데이터 모임에서 검색을 수행
- 이진 검색: 일정한 규칙으로 늘어놓은 데이터 모임에서 아주 빠른 검색을 수행
- 해시법: 추가, 삭제가 자주 일어나는 데이터 모임에서 아주 빠른 검색을 수행
 - ✓ 체인법: 같은 해시값의 데이터를 선형 리스트로 연결하는 방법
 - ✓ 오픈 주소법: 데이터를 위한 해시값이 충돌할 때 재해시하는 방법



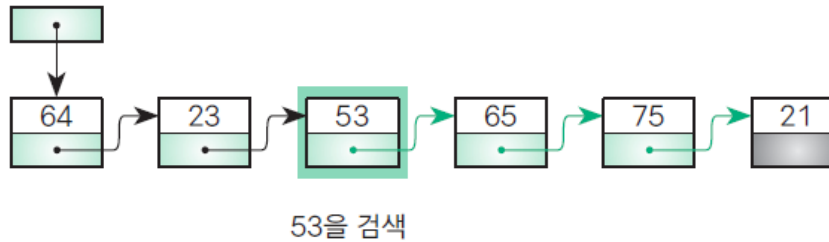
03-1 검색 알고리즘이란?

◆ 배열에서 검색하기

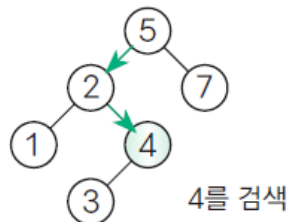
a 배열 검색



b 선형 리스트 검색



c 이진검색트리 검색



[그림 3-1] 검색이란 어떤 조건을 만족하는 데이터를 찾는 과정



03-2 선형 검색

선형 검색 다루기

- 배열에서 검색하는 방법 가운데 가장 기본적인 알고리즘은
선형 검색 linear search
- 선형 검색 linear search 또는 순차 검색 sequential search 라고 부름



03-2 선형 검색

◆ 선형 검색의 구체적인 과정 예시(1/2)

- 요소가 직선 모양으로 늘어선 배열에서의 검색은 원하는 키값을 갖는 요소를 만날 때까지 맨 앞부터 순서대로 요소를 검색
- **A** 값 2의 요소를 검색하는 데 성공한 예
- **B** 값 5의 요소를 검색하는 데 실패한 예
- 다음 페이지 그림 확인!

<선형 검색에서 배열 검색의 종료 조건>

- ① 검색할 값을 발견하지 못하고 배열의 끝을 지나간 경우(검색 실패)
- ② 검색할 값과 같은 요소를 발견한 경우(검색 성공)
- 배열 요소의 개수가 n 개일 때 ①, ② 조건을 판단하는 횟수는 평균 $n/2$ 회



03-2 선형 검색

◆ 선형 검색의 구체적인 과정 예시(2/2)

A 2를 검색: 검색 성공

	0	1	2	3	4	5	6
a	6	4	3	2	1	2	8
b	0	1	2	3	4	5	6
	6	4	3	2	1	2	8
c	0	1	2	3	4	5	6
	6	4	3	2	1	2	8
d	0	1	2	3	4	5	6
	6	4	3	2	1	2	8

검색 성공!
검색할 값과 같은 요소를 발견

B 5를 검색: 검색 실패

	0	1	2	3	4	5	6
a	6	4	3	2	1	2	8
b	0	1	2	3	4	5	6
	6	4	3	2	1	2	8
c	0	1	2	3	4	5	6
	6	4	3	2	1	2	8
d	0	1	2	3	4	5	6
	6	4	3	2	1	2	8
e	0	1	2	3	4	5	6
	6	4	3	2	1	2	8
f	0	1	2	3	4	5	6
	6	4	3	2	1	2	8
g	0	1	2	3	4	5	6
	6	4	3	2	1	2	8
h	0	1	2	3	4	5	6
	6	4	3	2	1	2	8

검색 실패! 배열의 끝을 지나갔습니다.

[그림 3-2] 선형 검색의 예(검색 성공 및 실패)



03-2 선형 검색

◆ 선형 검색 코드 대표 예시

■ 요소 개수가 n인 배열 a에서 값이 key인 요소를 검색하는 코드

- ✓ 배열을 검색할 때 배열 요소의 인덱스를 가리키는 변수는 i
- ✓ i는 0으로 초기화하고, 요소를 하나 검색할 때마다 while 문이 제어하는 루프 본문의 끝에서 증가
- ✓ 배열 검색 종료 조건 중 어느 하나라도 성립한 경우에는 while 문을 빠져나감

```
5 /*--- 요소의 개수가 n인 배열 a에서 key와 일치하는 요소를 선형 검색 ---*/
6 int search(const int a[], int n, int key)
7 {
8     int i = 0;
9     while (1) {
10         if (i == n)
11             return -1;    // 검색 실패
12         if (a[i] == key)
13             return i;    // 검색 성공
14         i++;
15     }
16 }
```

- 1

i == n이 성립하는 경우 (종료 조건 ①)
- OR
- 2

a[i] == key가 성립하는 경우 (종료 조건 ②)



03-2 선형 검색

◆ 선형 검색 - 예시 코드 1

■ 예제 3-1

- ✓ search 함수는 배열 a의 처음부터 끝까지 n개의 요소를 대상으로 값이 key인 요소를 선형 검색
- ✓ 반환값은 발견한 요소의 인덱스
- ✓ 만약 값이 key인 요소가 여러 개 존재한다면 반환 값은 검색 과정에서 처음 발견한 요소의 인덱스
- ✓ 값이 key인 요소가 존재하지 않으면 -1을 반환

전체 코드는 다음 장을 확인!



03-2 선형 검색

Chapt03_예제 3-1/ssearch1.c

// 선형 검색

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/*--- 요소의 개수가 n인 배열 a에서 key와 일치하는 요소를 선형 검색 ---*/
```

```
int search(const int a[], int n, int key)
```

```
{
```

```
    int i = 0; // 배열의 인덱스를 의미함
```

```
    while (1) {
```

```
        if (i == n)
```

```
            return -1; // 검색 실패
```

```
        if (a[i] == key)
```

```
            return i; // 검색 성공
```

```
        i++;
```

```
    }
```

```
}
```



03-2 선형 검색

```
int main(void)
{
    int nx, ky;
    puts("선형 검색");
    printf("요소 개수: ");
    scanf("%d", &nx);
    int* x = calloc(nx, sizeof(int)); //요소의 개수가 nx인 int형 배열 x를 생성
    for (int i = 0; i < nx; i++) {
        printf("x[%d] : ", i);
        scanf("%d", &x[i]);
    }
    printf("검색값: ");
    scanf("%d", &ky);
    int idx = search(x, nx, ky); // 배열 x의 값이 ky인 요소를 선형 검색
    if (idx == -1)
        puts("검색에 실패했습니다.");
    else
        printf("%d(은)는 x[%d]에 있습니다.\n", ky, idx);
    free(x); // 배열 x를 해제

    return 0;
}
```

실행 결과

선형 검색

요소 개수: 7

x[0]: 6

x[1]: 4

x[2]: 3

x[3]: 2

x[4]: 1

x[5]: 2

x[6]: 8

검색값: 2

2은(는) x[3]에 있습니다.



03-2 선형 검색

◆ 선형 검색 - 예시 코드 2

■ 예제 3-2

- ✓ 실습 3-1의 while 문을 for 문으로 수정한 프로그램
- ✓ while 문이 아니라 for 문으로 구현하면 프로그램은 더 짧고 간결해짐

```
5 /*--- 요소의 개수가 n인 배열 a에서 key와 일치하는 요소를 선형 검색(for 문)---*/
6 int search(const int a[], int n, int key)
7 {
8     for (int i = 0; i < n; i++)
9         if (a[i] == key)
10             return i;           // 검색 성공
11
12     return -1;                  // 검색 실패
13 }
```

전체 코드는 다음 장을 확인!



03-2 선형 검색

◆ 보조법으로 검색 다루기(1/2)

■ 보조법

- ✓ 종료 조건을 검사하는 비용을 반(50%)으로 줄이는 방법

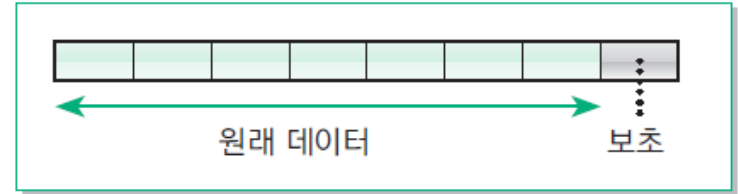
■ 보조법을 이용한 선형 검색

- ✓ 배열의 요소 $a[0] \sim a[6]$ 은 원래 데이터
- ✓ 맨 끝 요소 $a[7]$ 은 검색하기 전에 준비하는 보조
- ✓ 원하는 값이 원래의 데이터에 존재하지 않아도 보조인 $a[7]$ 까지 검색하면 종료조건 ②가 성립
- ✓ 이렇게 하면 원하는 키값을 찾지 못했을 때 종료 조건 ①이 없어도 됨
- ✓ 보조는 반복문에서 종료 판단 횟수를 2회에서 1회로 줄이는 역할을 함

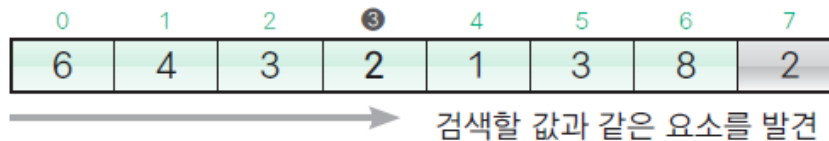


03-2 선형 검색

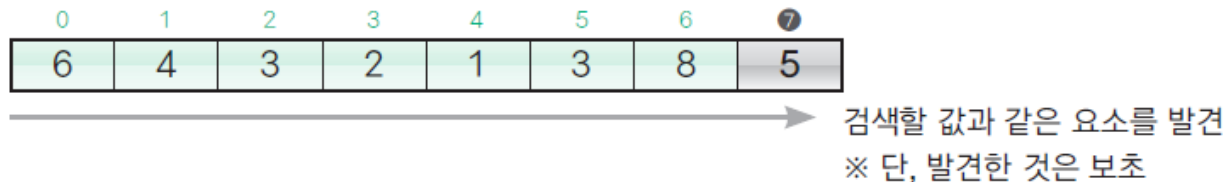
◆ 보조법으로 검색 다루기(2/2)



a 2를 검색(검색 성공)



b 5를 검색(검색 실패)



[그림 3-3] 보조법을 이용한 선형 검색

선형 검색에서 배열 검색의 종료 조건

- ① 검색할 값을 발견하지 못하고 배열의 끝을 지나간 경우(검색 실패)
- ② 검색할 값과 같은 요소를 발견한 경우(검색 성공)



03-2 선형 검색

◆ 보조법으로 검색 다루기 - 예시 코드 3

■ 예제 3 - 3

- ✓ **1** 검색값 key를 보조로 a[n]에 대입
- ✓ **2** 배열 요소를 순서대로 검사하는데 종료 조건 ①이 필요하지 않기 때문에 하나의 if 문만 사용
- ✓ **3** while 문에 의한 반복이 완료되면 찾은 값이 배열의 원래 데이터인지 아니면 보조인지 판단종료 조건을 검사하는 비용을 반(50%)으로 줄이는 방법
 - 이러한 판단을 수행하는 것이 if 문과 동등한 조건식 $i == n ? -1 : i$
 - 이 식을 이용하면 조건 결과를 다음과 같이 반환
 - 변수 i값이 n일 때 → 찾은 값이 보조이므로 검색 실패임을 나타내는 -1을 반환
 - 변수 i값이 n이 아닐 때 → 찾은 값이 원래 데이터이므로 i값을 반환



03-2 선형 검색

Chapt03_예제 3-3/ssearch_sen.c

// 선형 검색(보초법)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

/*--- 요소의 개수가 n인 배열 a에서 key와 일치하는 요소를 선형 검색(보초법) ---*/

```
int search(int a[], int n, int key)
```

```
{
```

```
    int i = 0;
```

```
    a[n] = key;    // 보초를 추가
```

```
    while (1) {
```

```
        if (a[i] == key)
```

```
            break;    // 원하는 키값을 찾은 경우
```

```
        i++;
```

```
    }
```

```
    return i == n ? -1 : i; //배열에서 key 값을 찾았다면 그 위치(i)를, 찾지 못했다면 -1을 반환
```

```
}
```



03-2 선형 검색

```
int main(void)
{
    int nx, ky;

    puts("선형 검색(보초법);");
    printf("요소 개수: ");
    scanf("%d", &nx);
    int *x = calloc(nx + 1, sizeof(int)); // 요소의 개수가 (nx + 1)인 int형 배열을 생성
    for (int i = 0; i < nx; i++) { // 주의: 값을 읽은 것은 nx개
        printf("x[%d] : ", i);
        scanf("%d", &x[i]);
    }
    printf(" 검색값 : ");
    scanf("%d", &ky);
    int idx = search(x, nx, ky); // 배열 x에서 값이 ky인 요소를 선형 검색
    if (idx == -1)
        puts(" 검색에 실패했습니다. ");
    else
        printf(" %d(은)는 x[%d]에 있습니다.\n", ky, idx);
    free(x); // 배열 x를 해제

    return 0;
}
```

실행 결과

```
선형 검색(보초법)
요소 개수: 7
x[0]: 6
x[1]: 4
x[2]: 3
x[3]: 2
x[4]: 1
x[5]: 3
x[6]: 8
검색값: 3
3은(는) x[2]에 있습니다.
```



03-3 이진 검색

◆ 이진 검색 다루기(1/3)

■ 이진 검색 binary search

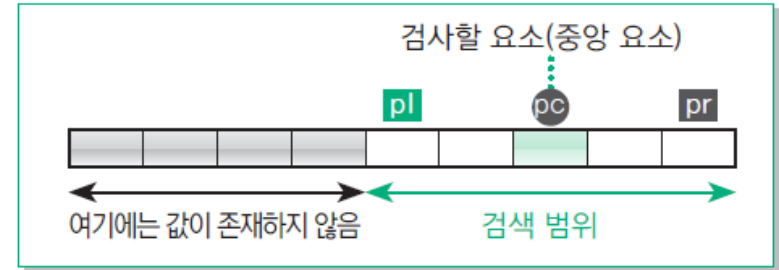
✓ 요소가 오름차순 또는 내림차순으로 정렬된 배열에서 검색하는 알고리즘

■ 이진 검색에서 배열 검색의 종료 조건

- ✓ ① 중앙 요소와 key가 일치하는 경우(검색 성공)
- ✓ ② 검색 범위가 더 이상 없는 경우(검색 실패)

■ 검색에 필요한 비교 횟수

- ✓ 이진 검색은 검색을 반복할 때마다 검색 범위가 절반이 되므로 검색에 필요한 비교 횟수의 평균값은 $\log n$
- ✓ 검색에 성공한 경우는 대략 $\log n - 1$ 회
- ✓ 검색에 실패한 경우는 $\lceil \log(n + 1) \rceil$ 회
 - $\lceil \rceil$ 는 천장 함수(ceiling function)를 나타내는 기호로 올림 함수라고도 함
 - $\lceil x \rceil$ 는 x의 천장 함수이며, x보다 크거나 같으면서 가장 작은 정수(예를 들어 $\lceil 3.5 \rceil$ 는 4)



<예시>

배열의 크기가 $n = 8$ 이라면:

첫 번째 비교 후: 배열의 크기 $8 \rightarrow 4$

두 번째 비교 후: 배열의 크기 $4 \rightarrow 2$

세 번째 비교 후: 배열의 크기 $2 \rightarrow 1$ (더 이상 나눌 수 없음)

따라서, 총 비교 횟수는 3번으로, $\log_2(8) = 3$ 이 됨



03-3 이진 검색

◆ 이진 검색 다루기(2/3)

■ 검색에 성공하는 예

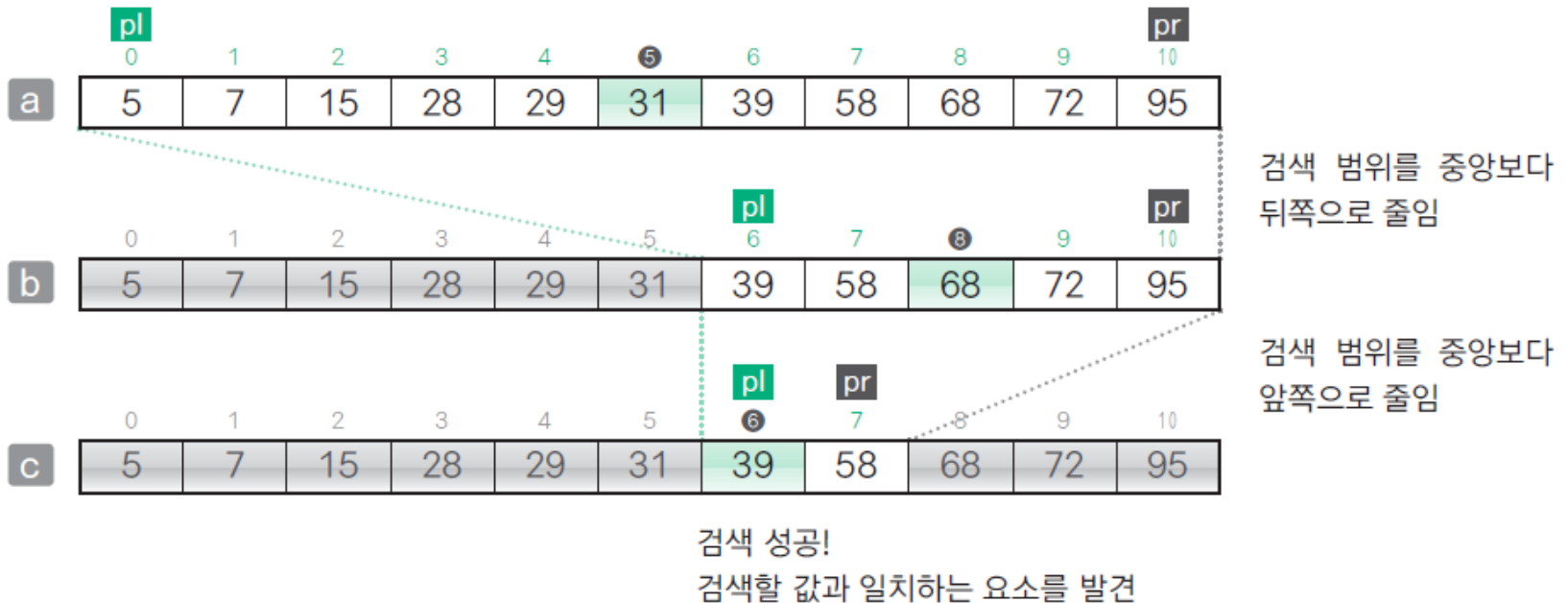
- ✓ **a** 검색 범위의 맨 앞 인덱스(0)를 pl, 맨 끝 인덱스(n-1)를 pr, 중앙 인덱스((n-1)/2)를 pc라고 지정
- ✓ **a** $\Rightarrow$ **b** $a[pc] < key$
 - $a[pl] \sim a[pc]$ 는 key보다 작은 것이 분명하므로 검색 대상에서 제외
 - 검색 범위는 중앙 요소 $a[pc]$ 보다 뒤쪽의 $a[pc+1] \sim a[pr]$ 로 좁혀짐
 - 그런 다음 pl의 값을 $pc + 1$ 로 업데이트
- ✓ **b** $\Rightarrow$ **c** $a[pc] > key$
 - $a[pc] \sim a[pr]$ 은 key보다 큰 것이 분명하므로 검색 대상에서 제외
 - 검색 범위는 중앙 요소 $a[pc]$ 보다 앞쪽의 $a[pl] \sim a[pc-1]$ 로 좁혀짐
 - 그런 다음 pr의 값을 $pc - 1$ 로 업데이트
- ✓ **c** $a[pc]$ 와 key를 비교하여 같으면 검색 성공

다음 장 그림을 참고!



03-3 이진 검색

◆ 이진 검색 다루기(2/3)



[그림 3-4] 이진 검색의 한 예(39를 검색: 검색 성공)

- 중앙값 $a[pc]$ 가 key보다 작을 때: 중앙의 다음 요소를 새로운 pl로 보고 뒤쪽으로 범위를 좁힙니다.
- 중앙값 $a[pc]$ 가 key보다 작을 때: 중앙의 이전 요소를 새로운 pr로 보고 앞쪽으로 범위를 좁힙니다.



03-3 이진 검색

◆ 이진 검색 다루기(3/3)

■ 검색에 실패하는 예

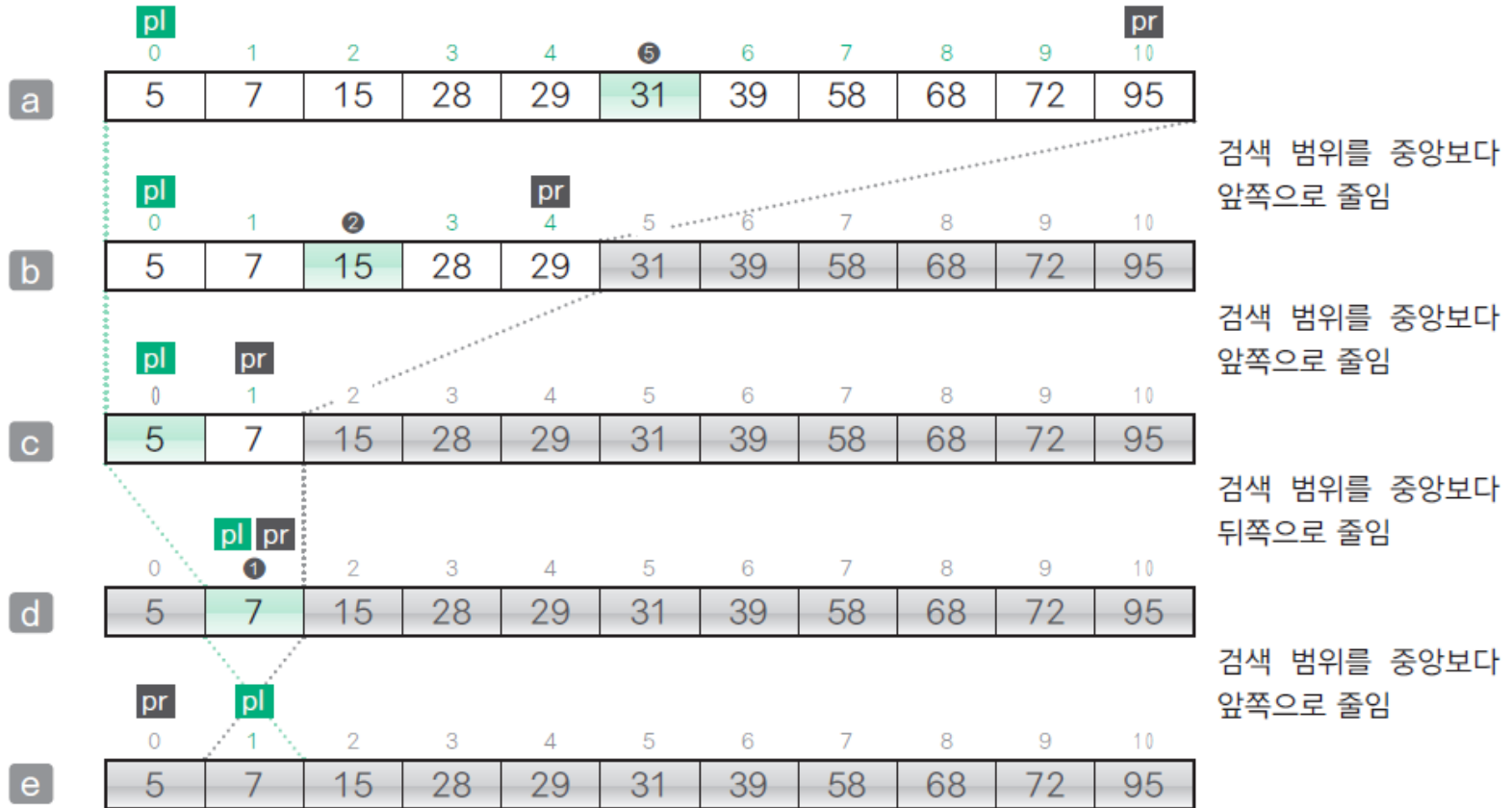
- ✓ **a** 검색할 범위는 모든 배열($a[0] \sim a[10]$)이고 중앙 요소 $a[5]$ 의 값 31이 키값인 6보다 크므로 검색 범위를 $a[0] \sim a[4]$ 로 좁힘
- ✓ **b** 새로 검색할 범위에서 중앙 요소 $a[2]$ 의 값 15가 키값인 6보다 크므로 검색할 범위를 $a[0] \sim a[1]$ 로 좁힘
- ✓ **c** 새로 검색할 범위에서 중앙 요소 $a[0]$ 의 값 5가 키값인 6보다 작으므로 pl 을 $pc + 1(1)$ 로 업데이트하면 pl 과 pr 은 1이 됨
- ✓ **d** 축소된 범위의 중앙 요소 $a[1]$ 의 값 7이 키값인 6보다 크므로 pr 을 $pc - 1(0)$ 로 업데이트하면 pl 이 pr 보다 커지면서 검색 범위를 더 이상 계산할 수 없게 되어 종료 조건 ②가 성립하므로 검색 실패

다음 장 그림을 참고!



03-3 이진 검색

◆ 이진 검색 다루기(3/3)



검색 실패! 검색 범위가 더 이상 없습니다.

[그림 3-5] 이진 검색의 한 예(6을 검색: 검색 실패)



03-3 이진 검색

◆ 이진 검색 다루기 - 예시 코드

■ 예제 3 - 4

- ✓ bin_search 함수는 요소의 개수가 n 인 배열 a 에서 key 와 일치하는 요소를 이진 검색
- ✓ 이진 검색을 구현한 프로그램
- ✓ 이진 검색 알고리즘은 검색 대상(배열)이 정렬(sort)되어 있음을 가정
- ✓ 따라서, 사용자가 각 요솟값을 입력하는 과정에서 바로 앞에 입력한 요소보다 작은 값인 경우에는 다시 입력하게 함

다음 장 코드를 참고!



03-3 이진 검색

Chapt03_예제 3-4/bin_search.c

// 이진 검색

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/*--- 요소의 개수가 n인 배열 a에서 key와 일치하는 요소를 이진 검색 ---*/
```

```
int bin_search(const int a[], int n, int key)
```

```
{
```

```
    int pl = 0;           // 검색 범위 맨 앞의 인덱스
```

```
    int pr = n - 1;       // 검색 범위 맨 끝의 인덱스
```

```
    do {
```

```
        int pc = (pl + pr) / 2; // 검색 범위 한가운데의 인덱스 → 반올림이 아닌 내림으로 계산
```

```
        if (a[pc] == key)      // 검색 성공
```

```
            return pc;
```

```
        else if (a[pc] < key)
```

```
            pl = pc + 1;       // 검색 범위를 뒤쪽 절반으로 좁힘
```

```
        else
```

```
            pr = pc - 1;       // 검색 범위를 앞쪽 절반으로 좁힘
```

```
    } while (pl <= pr);
```

```
    return -1;               // 검색 실패
```

```
}
```



03-3 이진 검색

```
int main(void)
{
    int nx, ky;

    puts("이진 검색");
    printf("요소 개수: ");
    scanf("%d", &nx);
    int *x = calloc(nx, sizeof(int)); // 요소 개수가 nx인 int형 배열 x 생성
    printf("오름차순으로 입력하세요.\n");
    printf("x[0] : ");
    scanf("%d", &x[0]);
    for (int i = 1; i < nx; i++) {
        do {
            printf("x[%d] : ", i);
            scanf("%d", &x[i]);
        } while (x[i] < x[i - 1]); // 바로 앞의 값보다 작으면 다시 입력
    }
    printf("검색값: ");
    scanf("%d", &ky);
    int idx = bin_search(x, nx, ky); // 배열 x에서 값이 ky인 요소를 이진 검색
    if (idx == -1)
        puts("검색에 실패했습니다.");
    else
        printf("%d는(은) x[%d]에 있습니다.\n", ky, idx);
    free(x); // 배열 x를 해제

    return 0;
}
```



03-3 이진 검색

◆ 복잡도

■ 복잡도 complexity

- ✓ 프로그램의 실행 속도는 프로그램이 동작하는 하드웨어나 컴파일러 등의 조건에 따라 달라짐
- ✓ 복잡도란 알고리즘의 성능을 객관적으로 평가하는 기준

■ 복잡도의 두 가지 요소

- ✓ 1. 시간 복잡도(time complexity): 실행에 필요한 시간을 평가한 것
- ✓ 2. 공간 복잡도(space complexity): 기억 영역과 파일 공간이 얼마나 필요한가를 평가한 것



03-3 이진 검색

◆ 복잡도 계산 예시(1/2)

- 양의 정수 n 을 n 번 더하는 문제를 생각하여 보자.

알고리즘 A	알고리즘 B	알고리즘 C
$\text{sum} \leftarrow n * n;$	for $i \leftarrow 1$ to n do $\text{sum} \leftarrow \text{sum} + n;$	for $i \leftarrow 1$ to n do for $j \leftarrow 1$ to n do $\text{sum} \leftarrow \text{sum} + 1;$



예를 들어 $n=3$ 일 때 $i=1 \sim 3$ 번 반복함
초기 : $\text{sum}=0$
1. $\text{sum}=3$
2. $\text{sum}=3+3$
3. $\text{sum}=3+3+3$



03-3 이진 검색

◆ 복잡도 계산 예시(2/2)

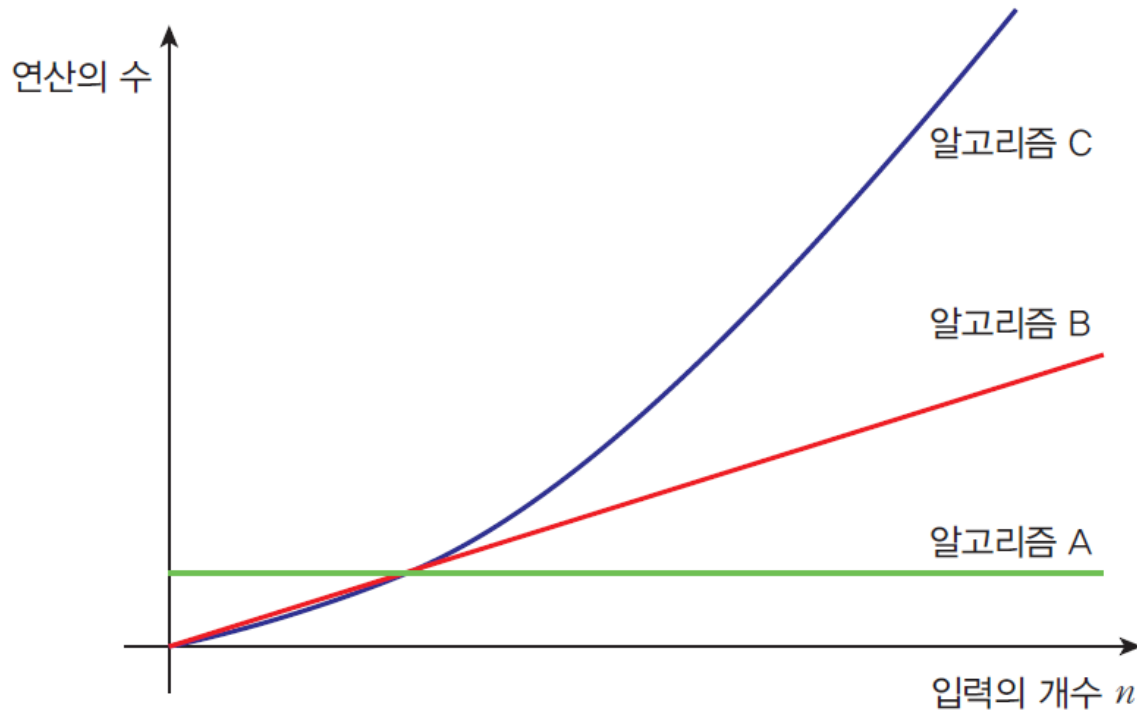
알고리즘 A	알고리즘 B	알고리즘 C
sum $\leftarrow$ n*n;	for i $\leftarrow$ 1 to n do sum $\leftarrow$ sum + n;	for i $\leftarrow$ 1 to n do for j $\leftarrow$ 1 to n do sum $\leftarrow$ sum + 1;

	알고리즘 A	알고리즘 B	알고리즘 C
대입연산	1	n	$n * n$
덧셈연산		n	$n * n$
곱셈연산	1		
나눗셈연산			
전체연산수	2	$2n$	$2n^2$



03-3 이진 검색

◆ 복잡도 계산 예시(2/2)





03-3 이진 검색

◆ 빅오표기법

- 자료의 개수가 많은 경우에는 차수가 가장 큰 항이 가장 영향을 크게 미치고 다른 항들은 상대적으로 무시될 수 있다.

$n=1000$ 인 경우

$$T(n) = n^2 + n + 1$$

99.9%

0.1%



03-3 이진 검색

◆ 빅오표기법

- 빅오 표기법: 연산의 횟수를 대략적(정근적)으로 표기한 것
- 빅오 표기법 특징
 - ✓ 상수항 무시: 빅오 표기법은 데이터 입력값(n)이 충분히 크다고 가정하여 알고리즘의 효율성(연산 횟수)을 평가하므로 상수항 같은 사소한 부분은 무시 (예: $2n \gg n$)
 - ✓ 영향력 없는 항 무시: 빅오 표기법은 데이터 입력값(n)의 크기에 따라 영향을 받기 때문에 가장 영향력이 큰 항 이외의 항은 무시 (예: $n^2 + 2n + 1 > n^2$)



03-3 이진 검색

◆ 빅오표기법의 종류

- $O(1)$: 상수형
- $O(\log n)$: 로그형
- $O(n)$: 선형
- $O(n \log n)$: 선형로그형
- $O(n^2)$: 2차형
- $O(n^3)$: 3차형
- $O(2^n)$: 지수형
- $O(n!)$: 팩토리얼형

$f(n)$	$O(f(n))$
10	$O(1)$
$5n^2 + 6$	$O(n^2)$
$2n^3 + 1$	$O(n^3)$
$2n^3 + 5n^2 + 6$	$O(n^3)$



03-3 이진 검색

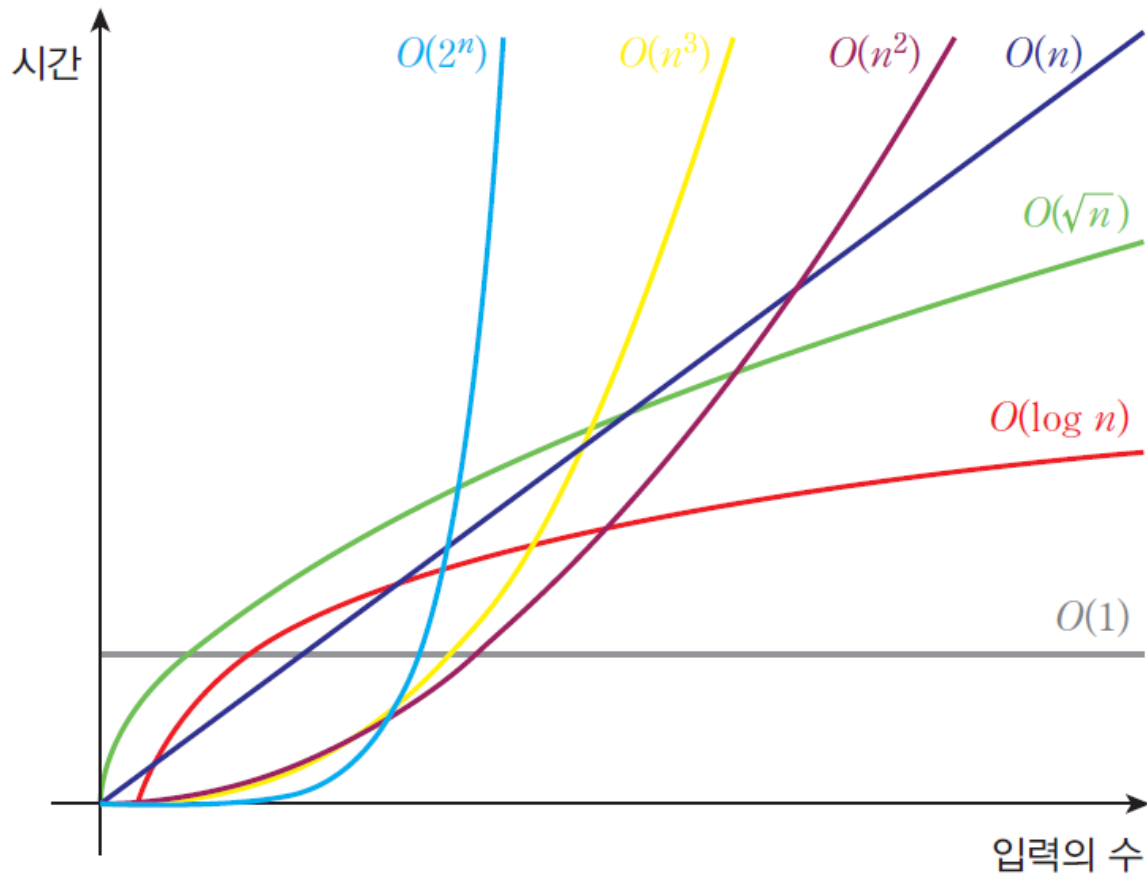
◆ 빅오표기법의 종류

시간복잡도	n					
	1	2	4	8	16	32
1	1	1	1	1	1	1
$\log n$	0	1	2	3	4	5
n	1	2	4	8	16	32
$n \log n$	0	2	8	24	64	160
n^2	1	4	16	64	256	1024
n^3	1	8	64	512	4096	32768
2^n	2	4	16	256	65536	4294967296
$n!$	1	2	24	40326	20922789888000	26313×10^{33}



03-3 이진 검색

◆ 빅오표기법의 종류





03-3 이진 검색

◆ 복잡도 살펴보기(1/3)

■ 선형 검색의 시간 복잡도 - (1)

- $O(1)$: n 과 무관하게 한 번만 실행하는 경우
 - 1 변수 i 에 0을 대입
 - 4 6 함수에서 값을 반환
- $O(n)$: n 에 비례하는 횟수만큼 실행하는 경우
 - 2 배열의 맨 끝에 도달했는지를 판단
 - 3 현재 검사하고 있는 요소와 찾고자 하는 값이 같은지를 판단
- n 이 점점 커지면 $O(n)$ 에 필요한 계산 시간은 n 에 비례하여 점점 길어짐
- 이와 달리 $O(1)$ 에 필요한 계산 시간은 변하지 않음

```

int search(const int a[], int n, int key)
{
1   int i = 0;
2   while(i < n) {
3       if(a[i] == key)
4           return i;      // 검색 성공
5       i++;
6   }
   return -1;      // 검색 실패
}

```

[표 3-1] 선형 검색에서 각 단계의 실행 횟수와 복잡도

단계	실행 횟수	복잡도
1	1	$O(1)$
2	$n / 2$	$O(n)$
3	$n / 2$	$O(n)$

단계	실행 횟수	복잡도
4	1	$O(1)$
5	$n / 2$	$O(n)$
6	1	$O(1)$



03-3 이진 검색

◆ 복잡도 살펴보기(2/3)

■ 선형 검색의 시간 복잡도 - (2)

✓ $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$

❖ ➔ 일반적으로 $O(f(n))$ 과 $O(g(n))$ 의 복잡도를 계산하는 방법임

✓ $O(1) + O(n) + O(n) + O(1) + O(n) + O(1) = O(\max(1, n, n, 1, n, 1)) = O(n)$

❖ 2개 이상의 복잡도로 구성된 알고리즘의 전체 복잡도는 차원이 더 높은 쪽의 복잡도를 우선시함

❖ 둘이 아니라 셋 이상의 계산으로 구성된 알고리즘도 마찬가지

❖ 다시 말해 전체 복잡도는 차원이 가장 높은 복잡도를 선택

❖ 그러므로 선형 검색 알고리즘의 복잡도를 구하면 $O(n)$ 이 됨



03-3 이진 검색

◆ 복잡도 살펴보기(3/3)

■ 이진 검색의 시간 복잡도

✓ $O(1) + O(1) + O(\log n) + O(\log n) + O(1) +$
 $O(\log n) + \dots + O(1) = O(\log n)$

❖ 이진 검색법을 이용하면 검색할 요소의 범위를 거의 절반씩 줄일 수 있으므로 복잡도가 $O(\log n)$ 이 됨

[표 3-2] 이진 검색에서 각 단계의 실행 횟수와 복잡도

단계	실행 횟수	복잡도
1	1	$O(1)$
2	1	$O(1)$
3	$\log n$	$O(\log n)$
4	$\log n$	$O(\log n)$
5	1	$O(1)$

```

int bin_search(const int a[], int n, int key)
{
1   int pl = 0;           // 검색 범위 맨 앞의 인덱스
2   int pr = n - 1;       // 검색 범위 맨 끝의 인덱스

   do {
3       int pc = (pl + pr) / 2; // 중앙 요소의 인덱스
4       if(a[pc] == key)
5           return pc;         // 검색 성공
6       else if(a[pc] < key)
7           pl = pc + 1;       // 검색 범위를 뒤쪽 반으로 줄임
8       else
9           pr = pc - 1;       // 검색 범위를 앞쪽 반으로 줄임
   } while(pl <= pr);

10  return -1;             // 검색 실패
}

```

6	$\log n$	$O(\log n)$
7	$\log n$	$O(\log n)$
8	$\log n$	$O(\log n)$
9	$\log n$	$O(\log n)$
10	1	$O(1)$

「2026년도 1학기 한국성서대학교 AI융합학부」

감사합니다.

2026.03.23

3주차

